

Machine Learning for Tabular Data

Fundamentals of Artificial Intelligence

	Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	LandContour	Utilities	...	PoolArea	PoolQC	Fence	MiscFeature	MiscVal	MoSold	YrSold	SaleType	SaleCondition	SalePrice
0	1	60	RL	65.0	8450	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	NaN	0	2	2008	WD	Normal	208500
1	2	20	RL	80.0	9600	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	NaN	0	5	2007	WD	Normal	181500
2	3	60	RL	68.0	11250	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN	NaN	0	9	2008	WD	Normal	223500
3	4	70	RL	60.0	9550	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN	NaN	0	2	2006	WD	Abnorml	140000
4	5	60	RL	84.0	14260	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN	NaN	0	12	2008	WD	Normal	250000
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
1455	1456	60	RL	62.0	7917	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	NaN	0	8	2007	WD	Normal	175000
1456	1457	20	RL	85.0	13175	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	MnPrv	NaN	0	2	2010	WD	Normal	210000
1457	1458	70	RL	66.0	9042	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	GdPrv	Shed	2500	5	2010	WD	Normal	266500
1458	1459	20	RL	68.0	9717	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	NaN	0	4	2010	WD	Normal	142125
1459	1460	20	RL	75.0	9937	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	NaN	0	6	2008	WD	Normal	147500

[1460 rows x 81 columns]

Supervised Learning in Mathematical Terms

map input to output: $y = f(\mathbf{x})$

random variables Y and $\mathbf{X} = (X_1, X_2, \dots, X_p) \leftarrow$ usually high-dimensional

training (e.g., curve fitting / parameter estimation):

fit data set of $(y_i, \mathbf{x}_i)$ pairs $\rightarrow$ minimize deviations between y and $f(\mathbf{x})$

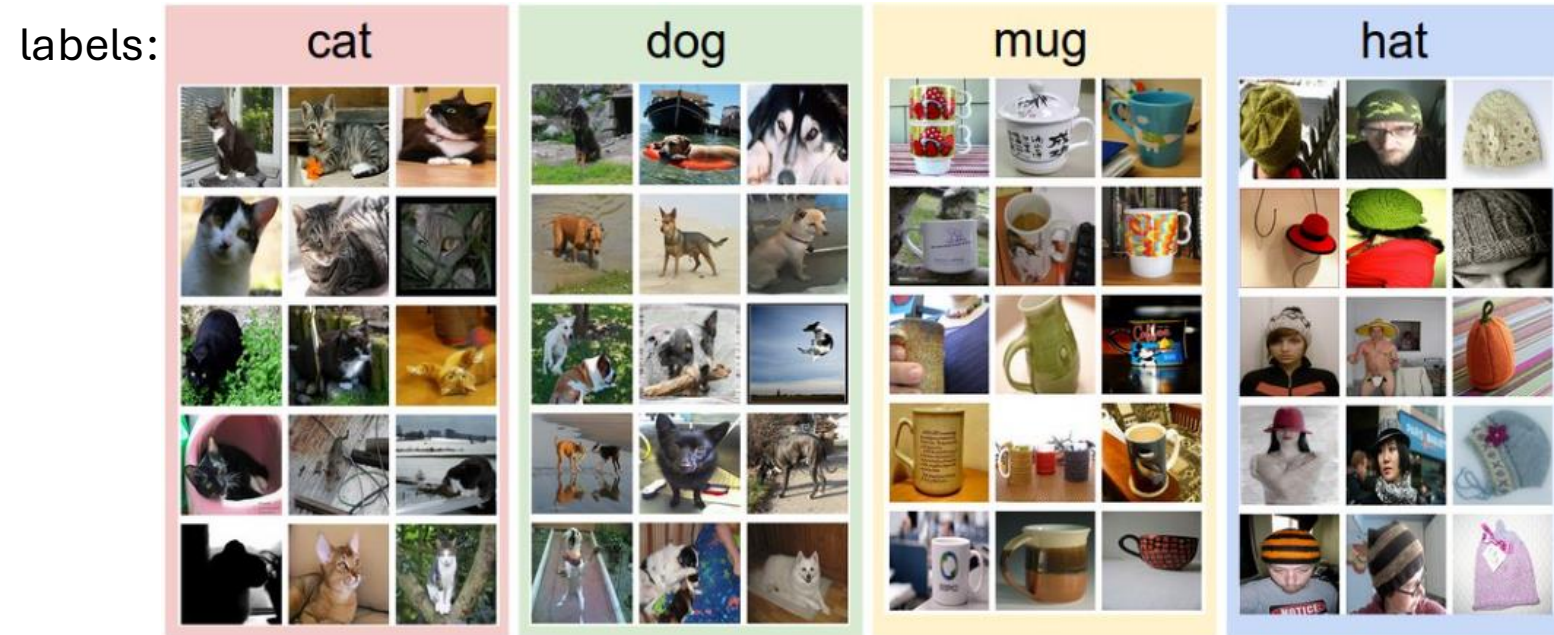
discriminative models: predict conditional density function $p(y|\mathbf{x})$

as opposed to generative models: predict $p(y, \mathbf{x})$ (or just $p(\mathbf{x})$) $\rightarrow \mathbf{x}$ not given, more difficult

Example: Image Classification

training data set:

test data with learned classifier:



$$f\left(\text{img of a cat}\right) = \text{"Cat"}$$

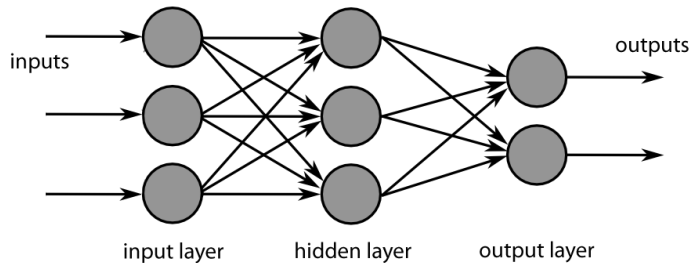
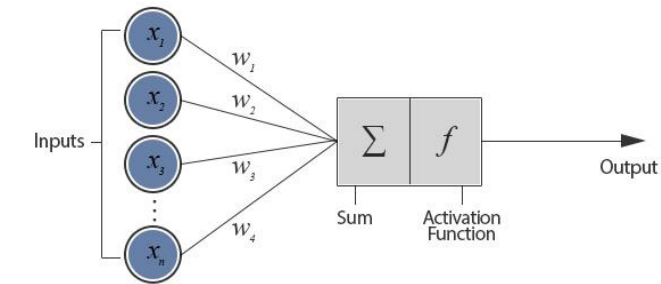
$$f\left(\text{img of a dog}\right) = \text{"Dog"}$$

Algorithmic Families of Supervised Learning

parametric models

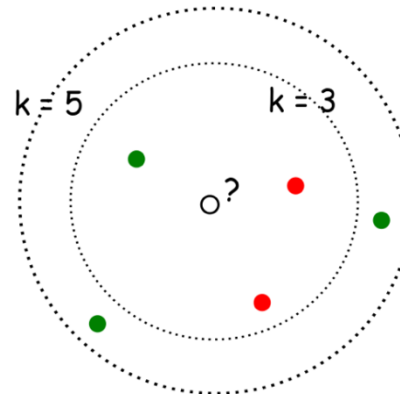
linear models

neural networks



non-parametric models: similarity-based learning (analogies)

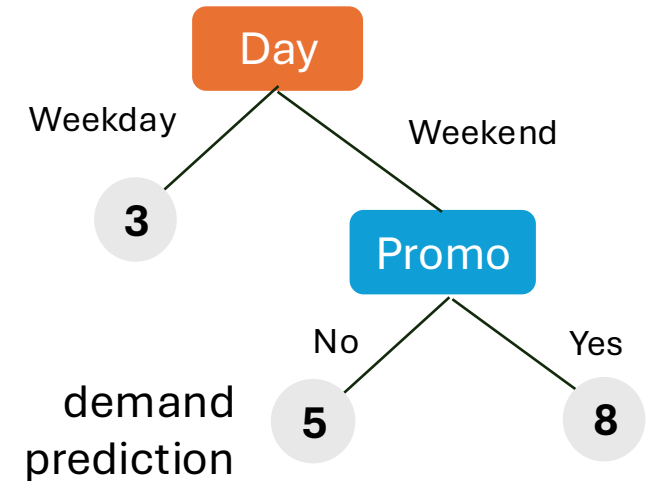
nearest-neighbor-style/local methods



with $k=3$, ●
with $k=5$, ●

kernel machines (e.g., support-vector machines)

decision trees: rule learning



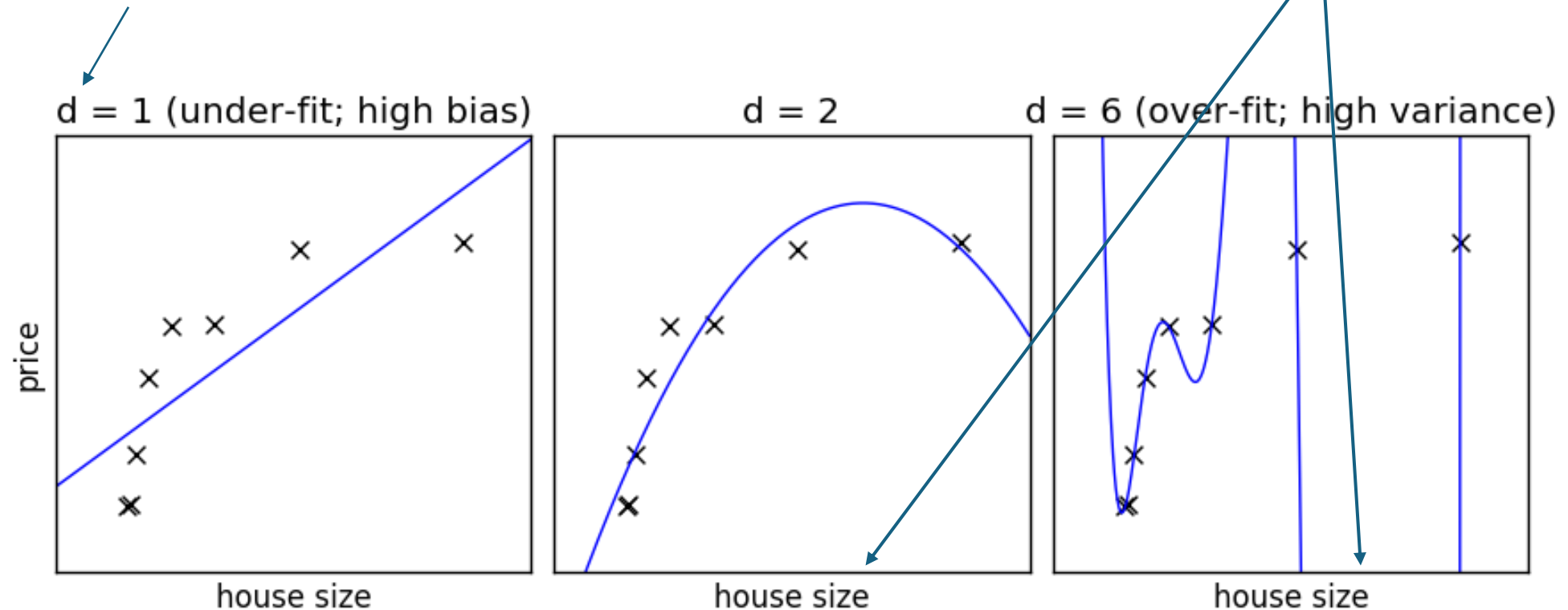
often used in ensemble methods

- bagging: random forests
- boosting: gradient boosting

Under- and Over-Fitting

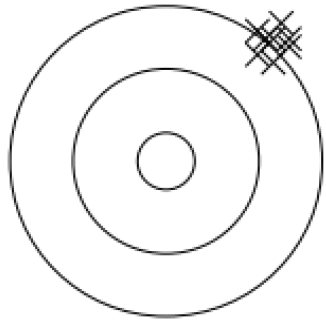
example: polynomial fit

degree of fitted polynomial



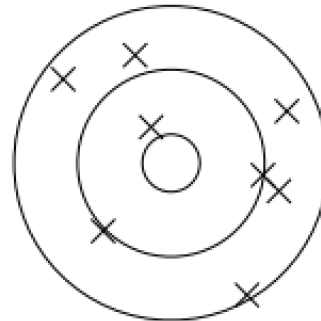
Bias, Variance, Irreducible Error

think of fitting ML algorithms as repeatable processes with different data sets



bias

due to too simplistic model
(same for all training data sets)
“underfitting”



variance

due to sensitivity to specifics (noise)
of different training data sets
“overfitting”

irreducible error (aka Bayes error):

inherent randomness (target generated from random variable)

→ limiting accuracy of ideal model

Bias-Variance Tradeoff

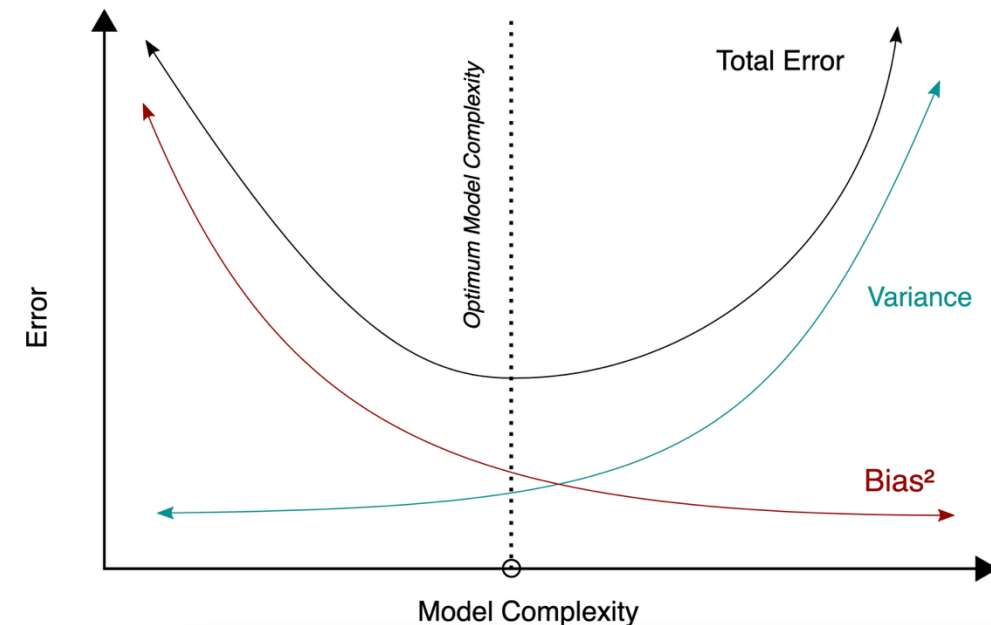
models of higher complexity have lower bias but higher variance
(given the same number of training examples)

generalization error follows U-shaped curve:
overfitting once model complexity (number of
parameters) passes certain threshold

overfitting: variance term dominating test error

→ increasing model complexity increases test error

but beware: training error keeps decreasing with
more complex model ...

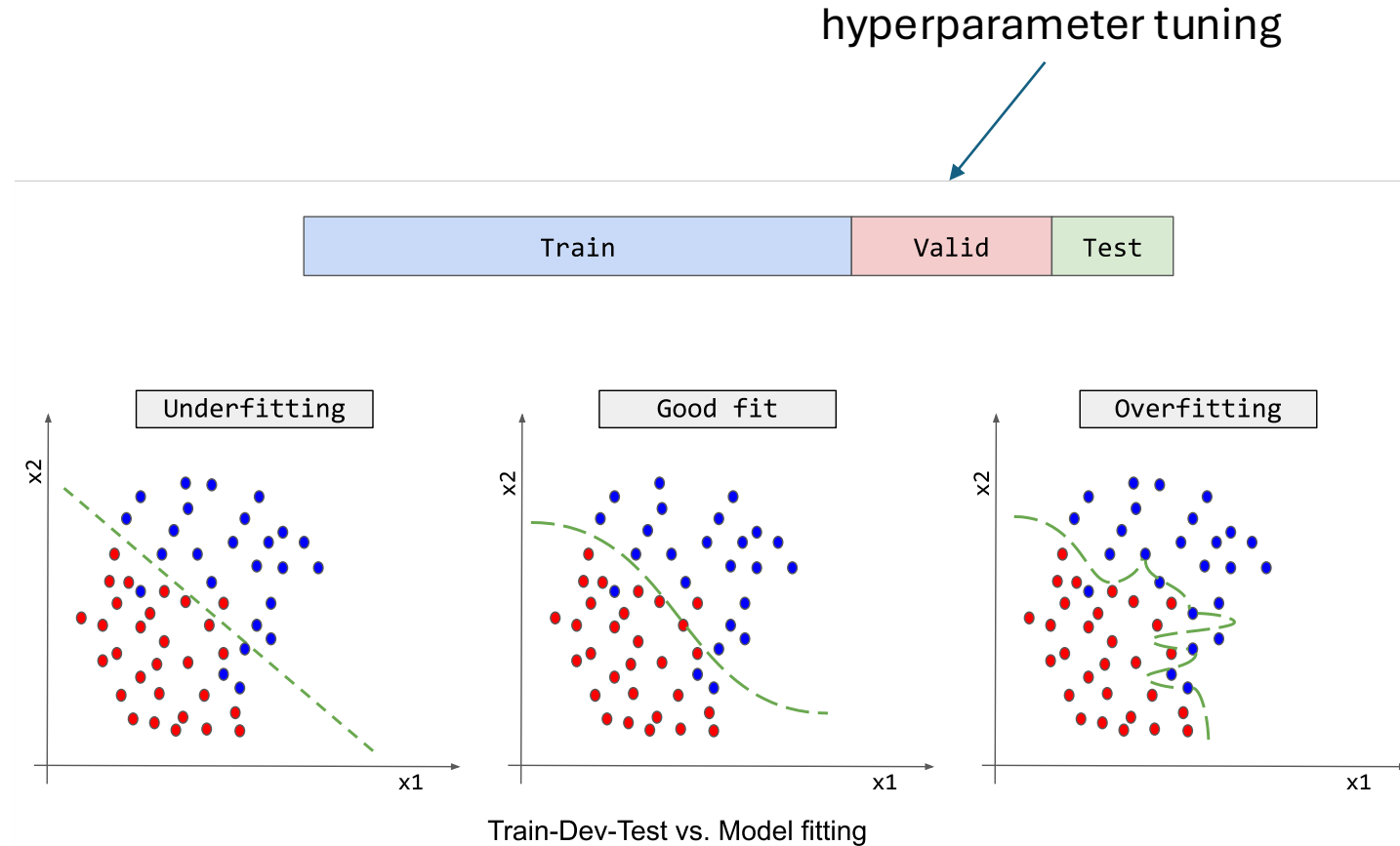


Hyperparameters

model complexity often controlled via hyperparameters (parameters not fitted but set in advance)

examples:

- degree of fitted polynomial d
- number of considered nearest neighbors k
- maximum depth of decision tree
- number of hidden nodes in neural network layer



Generalization

core of ML: **empirical risk minimization** (training error) as proxy for minimizing unknown population risk (test error)

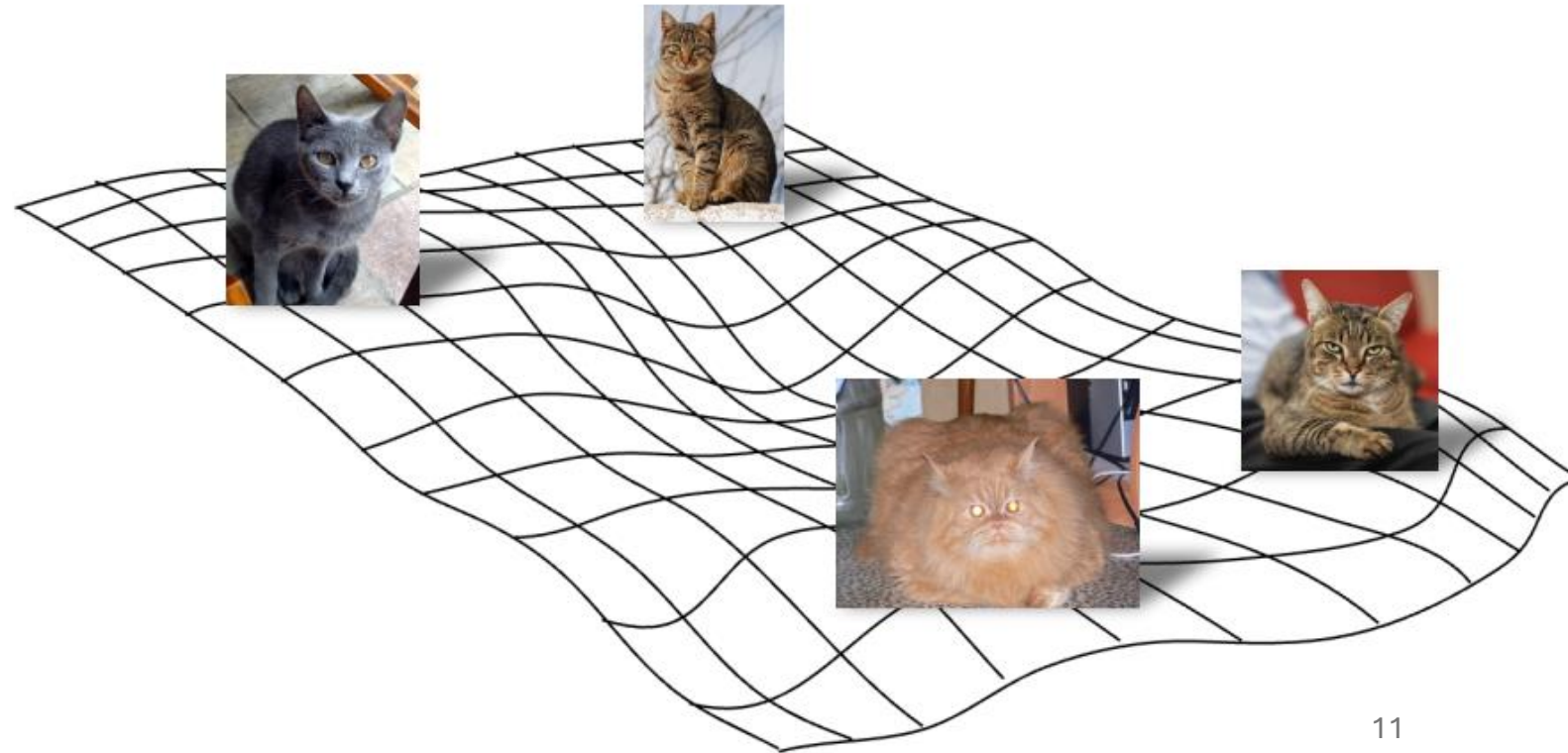
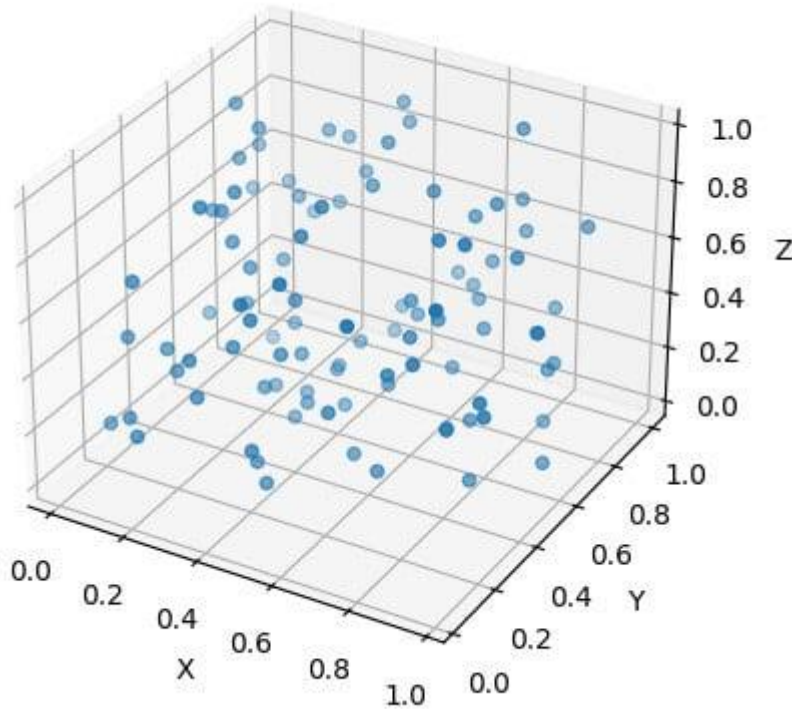
generalization gap: difference between test and training error

curse of dimensionality: typically, many features (dimensions)

→ lots of data needed to densely sample volume

Manifold Hypothesis

luckily, reality is friendly: most high-dimensional data sets reside on lower-dimensional manifolds → enabling effectiveness of ML



Linear Regression

fit:

$$y_i = b + \sum_{j=1}^p w_j x_{ij} + \varepsilon_i$$

predict:

$$\hat{y}_i = \mathbb{E}[Y|\mathbf{X} = \mathbf{x}_i] = f(\mathbf{x}_i) = b + \sum_{j=1}^p w_j x_{ij}$$

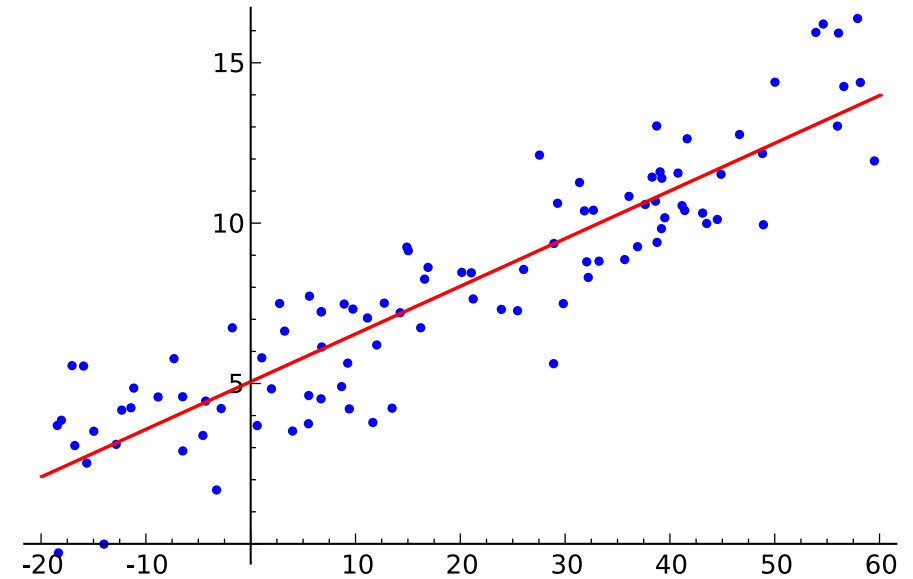
Gaussian

$$p(y|\mathbf{x}_i) = \mathcal{N}(y; \hat{y}_i, \sigma^2)$$

Gaussian

mean

variance



parameter estimation:
e.g., least squares

parameters to be estimated: $b, \mathbf{w}$
 $\rightarrow \sigma^2 = \frac{1}{n} \sum_{i=1}^n (y_i - f(\mathbf{x}_i))^2$

Classification: Logistic Regression

binary classification: $y = 1$ or $y = 0$

→ predict probabilities p_i for $y = 1$

- logit (log-odds) as link function
- Y following Bernoulli distribution

still Gaussian linear model

$$\text{logit}(\mathbb{E}[Y|\mathbf{X} = \mathbf{x}_i]) = \ln\left(\frac{p_i}{1 - p_i}\right) = \mathbf{w} \cdot \mathbf{x}_i + b$$


$$p_i = \frac{1}{1 + e^{-(\mathbf{w} \cdot \mathbf{x}_i + b)}}$$

Multi-Classification: Softmax

for classification in K categories:

- use “score” function with K outputs

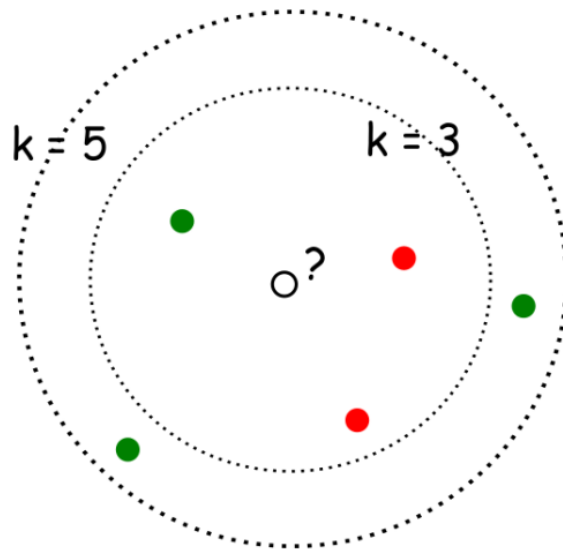
$$f_k(\mathbf{x}_i) = \mathbf{w}_k \cdot \mathbf{x}_i + b_k$$

- and convert into probabilities by means of softmax function

$$\frac{\exp(f_k(\mathbf{x}_i))}{\sum_{l=1}^K \exp(f_l(\mathbf{x}_i))}$$

k-Nearest Neighbors (kNN)

- local method, instance-based learning
- non-parametric
- distance defined by metric on $\mathbf{x}$ (e.g., Euclidean)



with $k=3$, ●
with $k=5$, ●

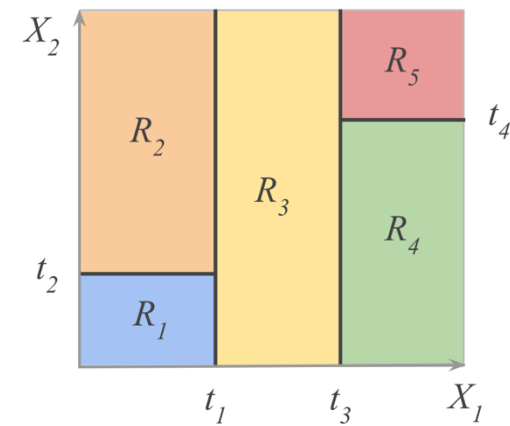
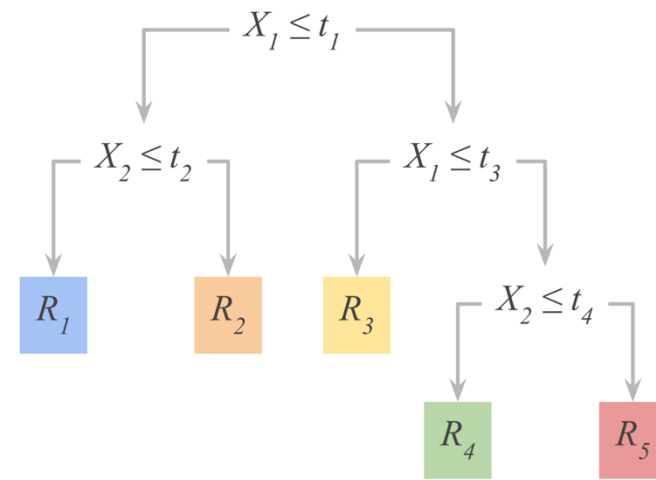
regression:

$$f(\mathbf{x}_0) = \frac{1}{k} \sum_{j=1}^k y_j \quad \text{with } j \text{ running over } k \text{ nearest neighbors of } \mathbf{x}_0$$

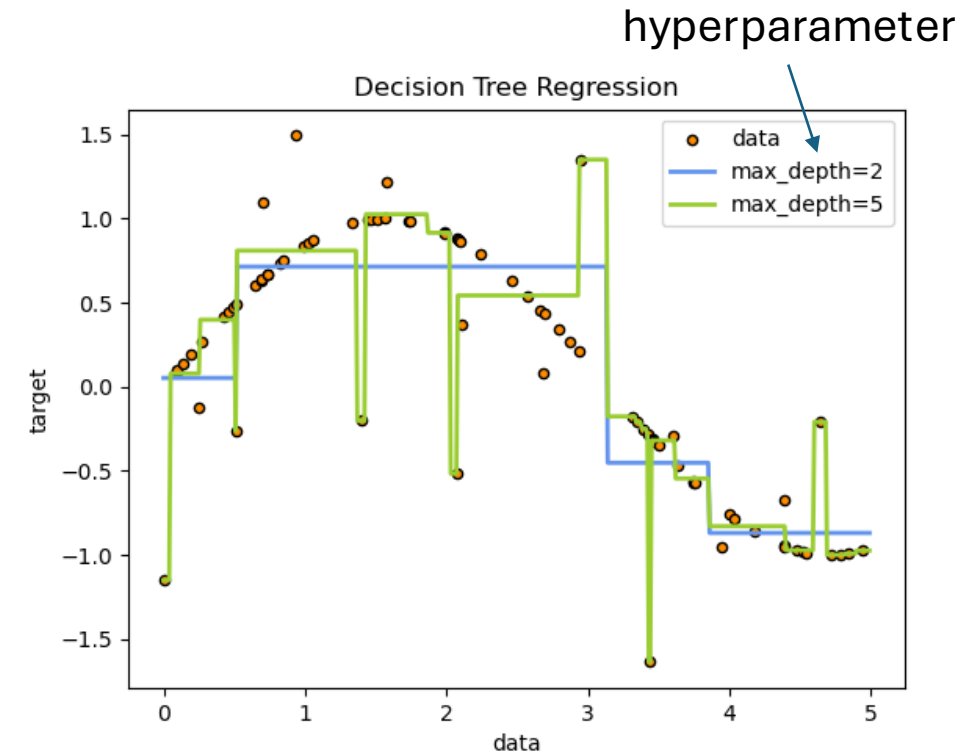
low k : low bias but high variance

high k : low variance but high bias

Decision Trees



- non-parametric learning of simple decision rules
- usually, binary trees
- axis-parallel decision boundaries (box-shaped regions in feature space)
- fit constant in each box
 - classification: majority class of target
 - regression: average of target
- fully explainable models



[scikit-learn](https://scikit-learn.org/)

Ensemble Learning

idea: combine several individual models (often of the same type) to form an ensemble model with better predictive performance than each of its constituents

→ learning in several different ways from the training data

- the trainings of the individual constituent models are (kind of) independent
- outputs of individual models are combined (e.g., averaged)

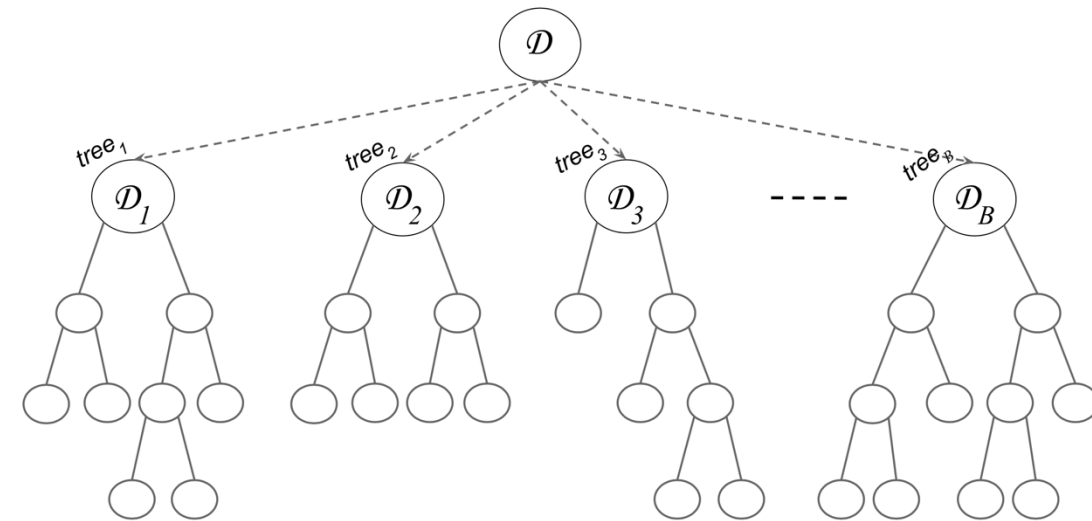
Bagging (Bootstrap Aggregating)

idea: train individual models of ensemble on random subsets of the training data (random sample with replacement)

- reduces variance of ensemble model compared to individual models
- but not bias $\rightarrow$ use low-bias base models

individual models combined by averaging (regression) or majority voting (classification) $\rightarrow$ committee of models

example with decision trees:



[source](#)

Random Subspace Method

aka feature bagging

idea: train different members (models) of an ensemble method on random subsets of all features (random sample with replacement)

→ reduce correlation between ensemble members

individual models combined by averaging (regression) or majority voting (classification) → committee of models

Random Forests

ensemble method using

- decision trees as individual models (usually, rather large, low-bias decision trees)
- combination of bagging and random subspace method (features sampled at each node in the trees)

compared to individual decision trees, random forests

+ reduce variance → improve accuracy

- lose explainability

one of the most popular off-the-shelf ML algorithms (often, good performance with little hyperparameter tuning and data preparation)

Boosting

idea: sequentially learn and combine several “weak” learners (such as small decision trees, but in principle any ML algorithm) to construct a “strong” one

→ gradually (in a greedy fashion) reducing bias of ensemble model

in simple terms:

building a model from the training data, then creating a second model that attempts to correct the errors from the first model, ...

→ each subsequent weak learner is forced to concentrate on the examples that are missed by the previous ones in the sequence

not a committee of models:

typically, use simple, high-bias methods as individual models (e.g., small decision trees)

most prominent: AdaBoost, Gradient Boosting

Forward Stagewise Additive Modeling

boosting can be understood as fitting an additive expansion in a set of basis functions (e.g., decision trees): $f(\mathbf{x}) = \sum_{m=1}^M \gamma_m h_m(\mathbf{x})$

loss minimization of $f(\mathbf{x})$ computationally expensive $\rightarrow$ sequentially add and fit individual basis functions without changing already fitted ones

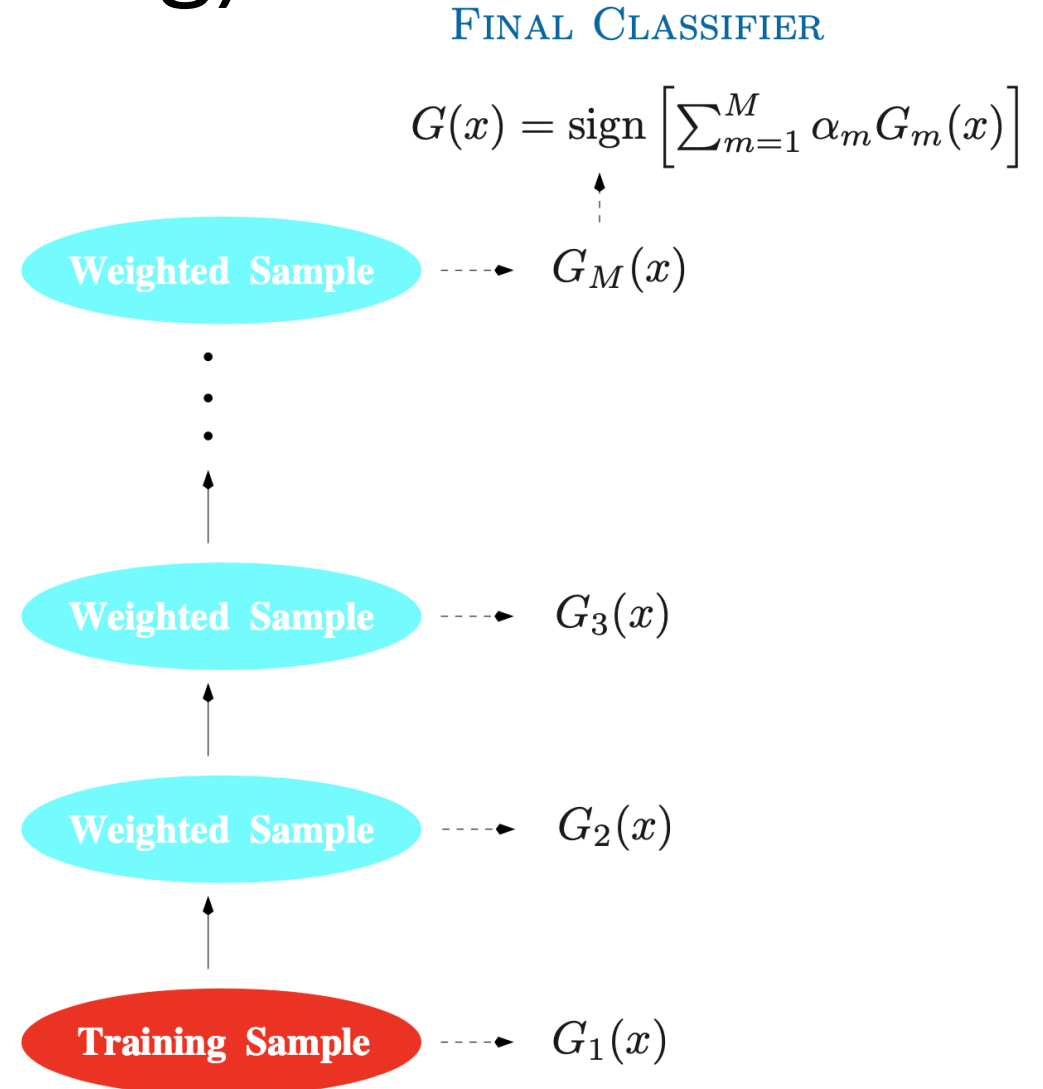
boosting algorithm:

for $m = 1$ to M :

1. compute $\underset{h_m, \gamma_m}{\operatorname{argmin}} \sum_{i=1}^n L(y_i, f_{m-1}(x_i) + \gamma_m h_m(x_i))$ (initialized with $f_0(\mathbf{x}) = 0$)
2. set $f_m(\mathbf{x}) = f_{m-1}(\mathbf{x}) + \gamma_m h_m(\mathbf{x})$

AdaBoost (Adaptive Boosting)

- iteratively train weak classifiers on weighted data
- emphasizing misclassified points (weights increased for incorrect predictions, decreased for correct ones)
- combine them into weighted vote (forward stagewise additive modeling)



Gradient Boosting

cost function

loss function

define $\mathcal{L}(f_m) = \sum_{i=1}^n L(y_i, f_{m-1}(x_i) + \gamma_m h_m(x_i))$

in forward stagewise additive modeling:

$$f_m(\mathbf{x}) = \sum_{m=1}^M \gamma_m h_m(\mathbf{x})$$

$$\operatorname{argmin}_{h_m, \gamma_m} \sum_{i=1}^n L(y_i, f_{m-1}(x_i) + \gamma_m h_m(x_i)) \quad \text{for each stage}$$

→ can be hard to solve for general loss function L

idea: approximation of $\mathcal{L}$ by its Taylor expansion

$$\mathcal{L}(f_m) \approx \mathcal{L}(f_{m-1}) + \sum_{i=1}^n h_m(x_i) \cdot \left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)} \right]_{f=f_{m-1}}$$

gradient g at x_i :

constant for this iteration

→ decreased by going a step of size γ_m (potentially optimized via line search) in the direction $h_m(\mathbf{x}) \approx -g$ (functional gradient descent)

→ find h_m by fitting a regression model with targets $-g$ (typically decision tree regressors of fixed size: gradient-boosted decision trees)

works like this for both regression and classification tasks (just different loss functions)

Decision Tree Sizes for Boosting

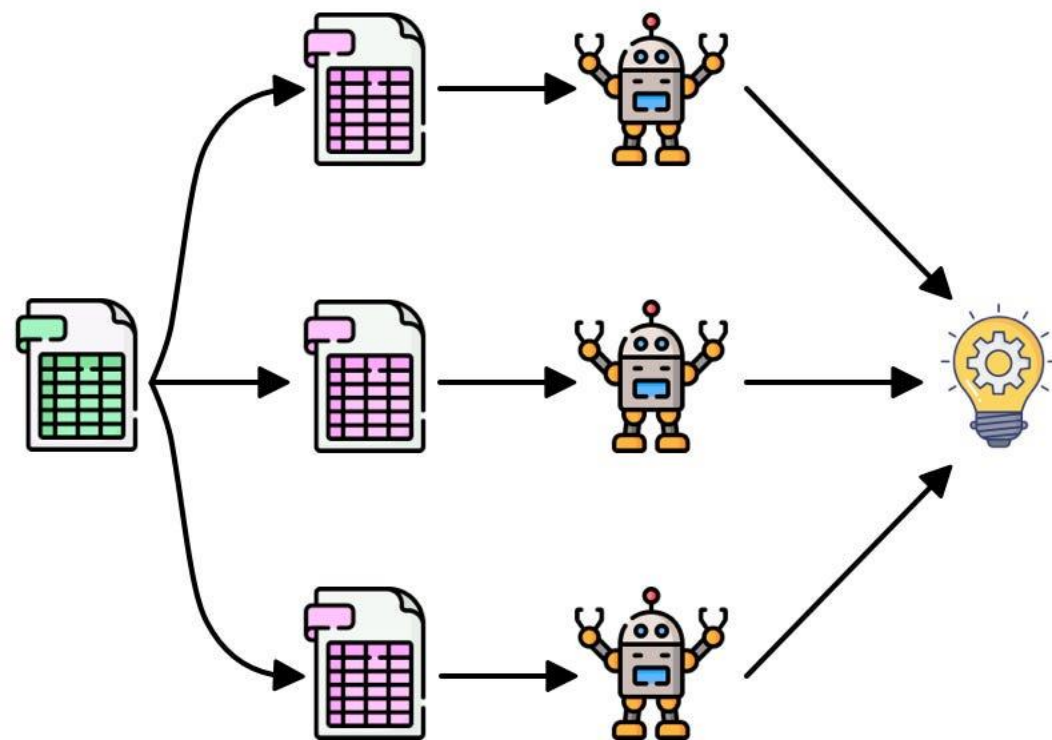
consider degree to which features interact with each other in given data set

- decision trees with single split (aka decision stumps): covering no interaction effects, just main effects of individual features
- decision trees with two splits: covering second-order interactions
- decision trees with three splits: covering third-order interactions

(usually, interaction order not much higher than ~ 5 in real-world data sets)

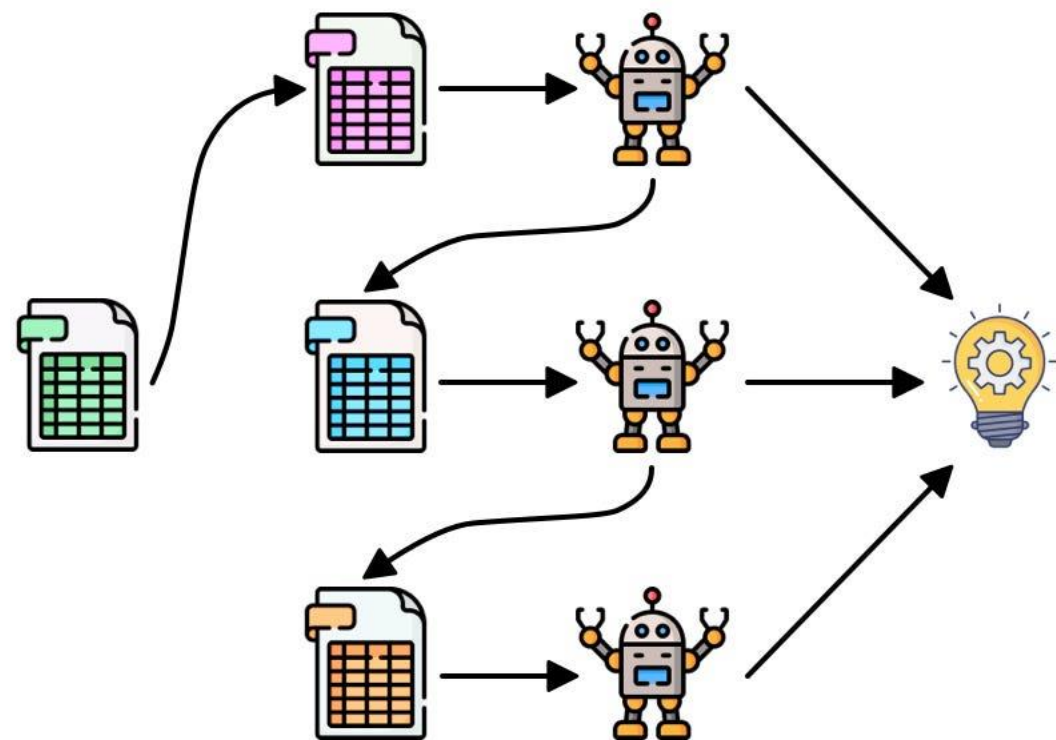
why boosting often works better than single large, low-bias model:
uncorrelated learners, each focusing on a specific aspect of the data

Bagging



Parallel

Boosting



Sequential

Practice Project

[Kaggle competition: Spaceship Titanic](#)